

# 企业级应用架构

# 目录

- 01 概述**
- 02 企业级应用架构演变**
- 03 企业级应用架构设计原则**
- 04 常见的软件开发模式**

# 01



## 概述

# 企业级应用的定义

企业级应用是指那些为商业组织、大型企业而创建的解决方案及应用程序。这些大型的企业级应用具有**用户数多、数据量大、事务密集**等特点，往往能够满足未来业务需要的变化，易于升级和维护。

一个好的企业级应用体系结构，通常来自优秀的解决方案，同时自应用程序设计的开始就要考虑其体系结构的合理性、灵活性、健壮性，从而既能满足企业级应用的复杂需求也能为今后系统的调整和升级留有余地。这样处理实际上是延长整个应用的生命周期，增强了用户在多变的商业社会中的适应性，减少了系统维护的开销和难度，从而给用户带来最大的利益。

当代的企业级应用决不可能是一个个相互独立的系统。在企业中，一般都会部署多个彼此连接的、相互通过不同集成层次进行交互的企业级应用，同时这些应用又都有可能与其它企业的相关应用连接，从而构成一个结构复杂的、跨越Intranet和Internet的分布式企业应用群集。

# 企业级应用程序特点

**基于网络的应用。**企业级的应用程序，规模应用如此巨大，一定不是集成在一两台及其之上，它的应用应该是基于一个范围更为宽广的网络之上，所以必定要有网络方面的应用需求。

**巨量的数据局集成。**在这样的一个应用当中，存放着企业的巨量信息，这些信息会作为数据库中的数据被保存起来。其中还包括之前应用系统的数据资源。

**高度的安全性。**由于是基于网络的应用，而数据又都是具有至少商业级的保密要求，因此，提供一个可靠、安全、稳定的系统是必不可少的特性之一。

**具备可扩展性。**当前的应用发展非常迅速，对于企业级应用软件，所服务的对象也是海量的用户，对于用户群里不断增加的新的需求，必须要具备一定的扩展性来适应用户不断提出的新的需求。

# 企业级应用相关概念

## 系统与子系统

**系统**泛指由一群有**关联**的个体组成，根据某种**规则**运作，能完成个别元件不能单独完成的工作的群体。它的意思是“总体”“整体”或“联盟”。（维基百科）

**关联**：系统是由一群有关联的个体组成的，没有关联的个体堆在一起不能成为一个系统。例如，把一个发动机和一台PC放在一起不能称之为一个系统，把发动机、底盘、轮胎、车架组合起来才能成为一台汽车。

**规则**：系统内的个体需要按照指定的规则运作，而不是单个个体各自为政。规则规定了系统内个体分工和协作的方式。例如，汽车发动机负责产生动力，然后通过变速器和传动轴，将动力输出到车轮上，从而驱动汽车前进。

**能力**：系统能力与个体能力有本质的差别，系统能力不是个体能力之和，而是产生了新的能力。例如，汽车能够载重前进，而发动机、变速器、传动轴、车轮本身都不具备这样的能力。

# 企业级应用相关概念

## 系统与子系统

**子系统**也是由一群有关联的个体所组成的系统，多半会是更大系统中的一部分。

其实子系统的定义和系统定义是一样的，只是观察的角度有差异，一个系统可能是另外一个更大系统的子系统。

按照这个定义，系统和子系统比较容易理解。我们以微信为例来做一个分析。

- 微信本身是一个系统，包含聊天、登录、支付、朋友圈等子系统。
- 朋友圈这个系统又包括动态、评论、点赞等子系统。
- 评论这个系统可能又包括防刷子系统、审核子系统、发布子系统、存储子系统。
- 评论审核子系统不再包含业务意义上的子系统，而是包括各个模块或者组件，这些模块或者组件本身也是另外一个维度上的系统。例如，MySQL、Redis等是存储系统，但不是业务子系统。

# 企业级应用相关概念

## 模块与组件

**软件模块 (Module)** 是一套一致而互相有紧密关连的软件组织。它分别包含了程序和数据结构两部分。现代软件开发往往利用模块作为合成的单位。模块的接口表达了由该模块提供的功能和调用它时所需的元素。模块是可能分开被编写的单位。这使它们可再用和允许人员同时协作、编写及研究不同的模块。（维基百科）

**软件组件 (component)** 定义为自包含的、可编程的、可重用的、与语言无关的软件单元，软件组件可以很容易被用于组装应用程序中。（维基百科）

**模块和组件都是系统的组成部分，只是从不同的角度拆分系统而已。**

# 企业级应用相关概念

## 模块与组件

模块和组件两个概念在实际工作中很容易混淆，我们经常能够听到类似这样的说法：

- MySQL模块主要负责存储数据，而ElasticSearch模块主要负责数据搜索。
- 我们有安全加密组件、有审核组件。
- App的下载模块使用了第三方的组件。

从**逻辑的角度**来拆分系统后，得到的单元就是“**模块**”；

从**物理的角度**来拆分系统后，得到的单元就是“**组件**”。

划分模块的主要目的是职责分离；划分组件的主要目的是单元复用。其实，“组件”的英文component也可翻译成中文的“零件”一词，“零件”更容易理解一些，“零件”是一个物理的概念，并且具备“独立且可替换”的特点。

我以一个最简单的网站系统来为例。假设我们要做一个学生信息管理系统，这个系统从逻辑的角度来拆分，可以分为“登录注册模块”“个人信息模块”“个人成绩模块”；

从物理的角度来拆分，可以拆分为Nginx、Web服务器、MySQL。

# 企业级应用相关概念

## 框架与架构

**软件框架 (Software framework)** 通常指的是为了实现某个业界标准或完成特定基本任务的软件组件规范，也指为了实现某个软件组件规范时，提供规范所要求之基础功能的软件产品。

**框架是组件规范**：例如，MVC就是一种最常见的开发规范，类似的还有MVP、MVVM、J2EE等框架。

**框架提供基础功能的产品**：例如，Spring MVC是MVC的开发框架，除了满足MVC的规范，Spring提供了很多基础功能来帮助我们实现功能，包括注解（@Controller等）、Spring Security、Spring JPA等很多基础功能。

# 企业级应用相关概念

## 框架与架构

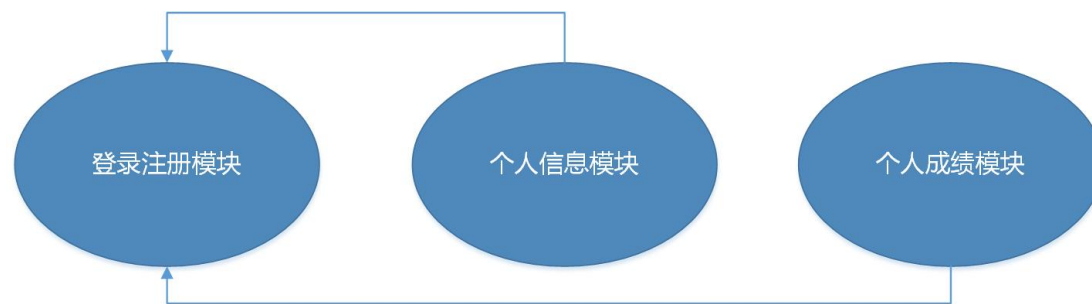
**软件架构**指软件系统的“基础结构”，创造这些基础结构的准则，以及对这些结构的描述。

单纯从定义的角度来看，框架和架构的区别还是比较明显的，框架关注的是“规范”，架构关注的是“结构”。框架的英文是Framework，架构的英文是Architecture。Spring MVC的英文文档标题就是“Web MVC framework”。

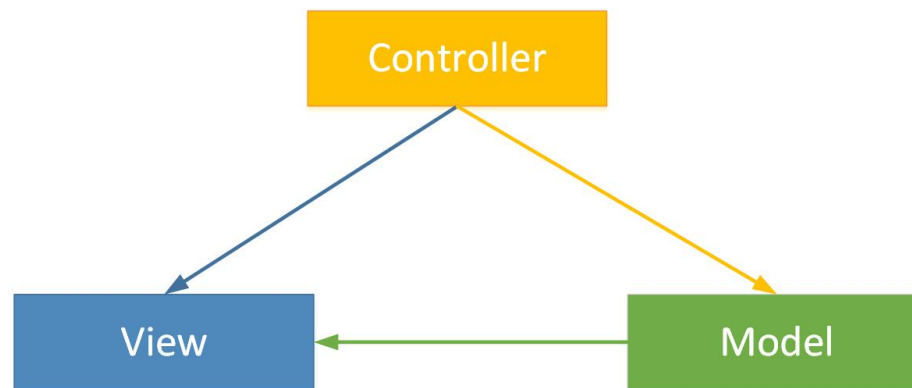
# 学生管理系统架构



从物理部署的角度分解



从业务逻辑的角度分解



从业务逻辑的角度分解

# 如何理解架构

## 软件架构指软件系统的顶层结构。

这个定义看似很简单，但包含的信息很丰富，基本上把系统、子系统、模块、组件、架构等概念都串起来了。

- 首先，“系统是一群关联个体组成”，这些“个体”可以是“子系统”“模块”“组件”等；架构需要明确系统包含哪些“个体”。
- 其次，系统中的个体需要“根据某种规则”运作，架构需要明确个体运作和协作的规则。
- 第三，“顶层结构”，可以更好地区分系统和子系统，避免将系统架构和子系统架构混淆在一起导致架构层次混乱。

# 什么是应用架构?

完整的企业架构（EA，Enterprise Architecture），包括业务架构、应用架构、技术架构、数据架构。  
应用架构作为信息系统建设的蓝图，起到了承上启下的作用，向上承接企业战略和商业模式，向下指导企业信息系统的建设。



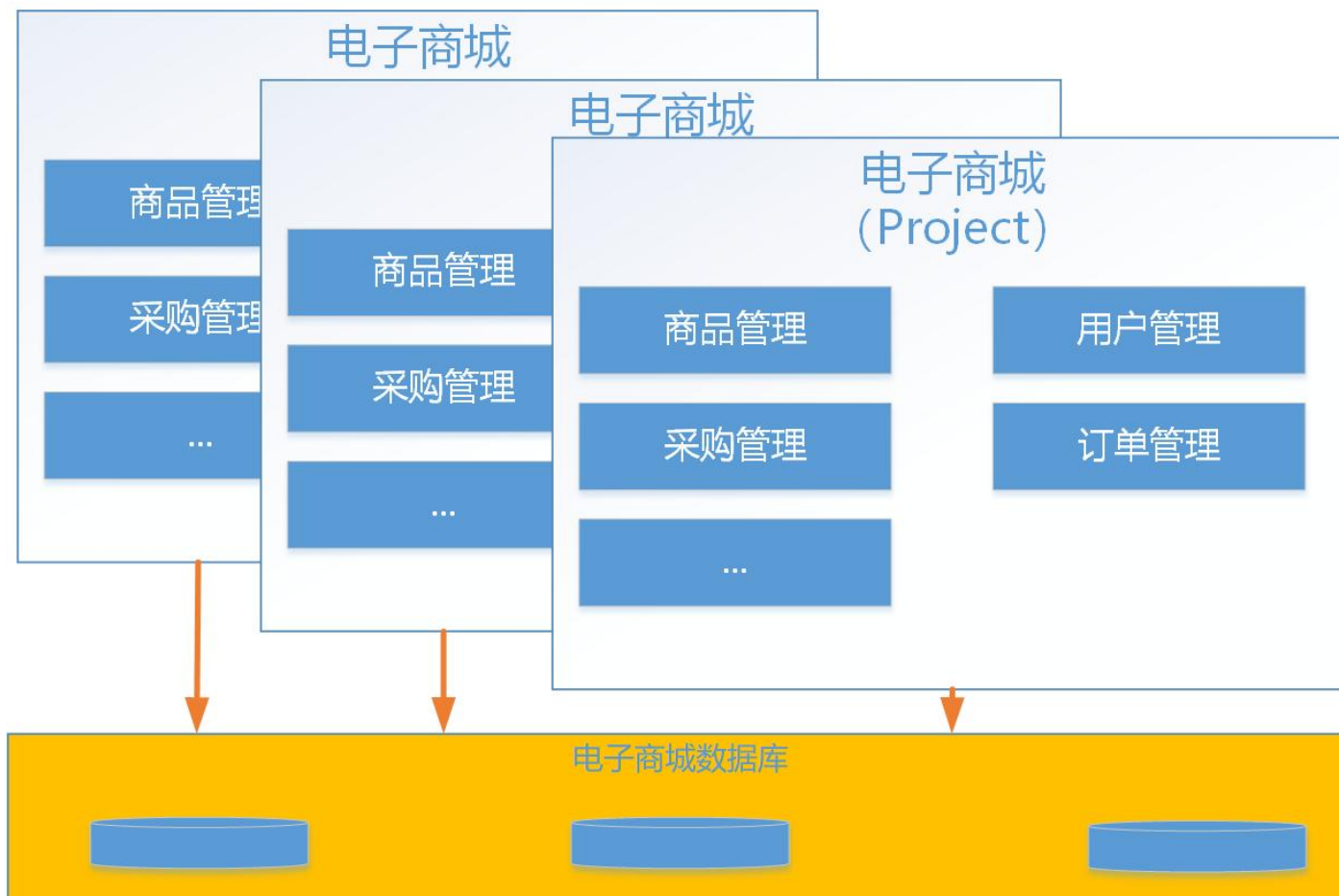
# 02

## 企业级应用架构演变

# 单体架构

Web应用程序发展的早期，大部分web工程师将所有的功能模块打包到一起并放在一个web容器中运行，所有功能模块使用同一个数据库。

右图为一个单体架构的电商系统



# 单体架构

## 特点：

- 1、所有的功能集成在一个项目工程中。
- 2、所有的功能打在一个war包部署到服务器。
- 3、通过部署应用集群和数据库集群来提高系统的性能。

## 优点：

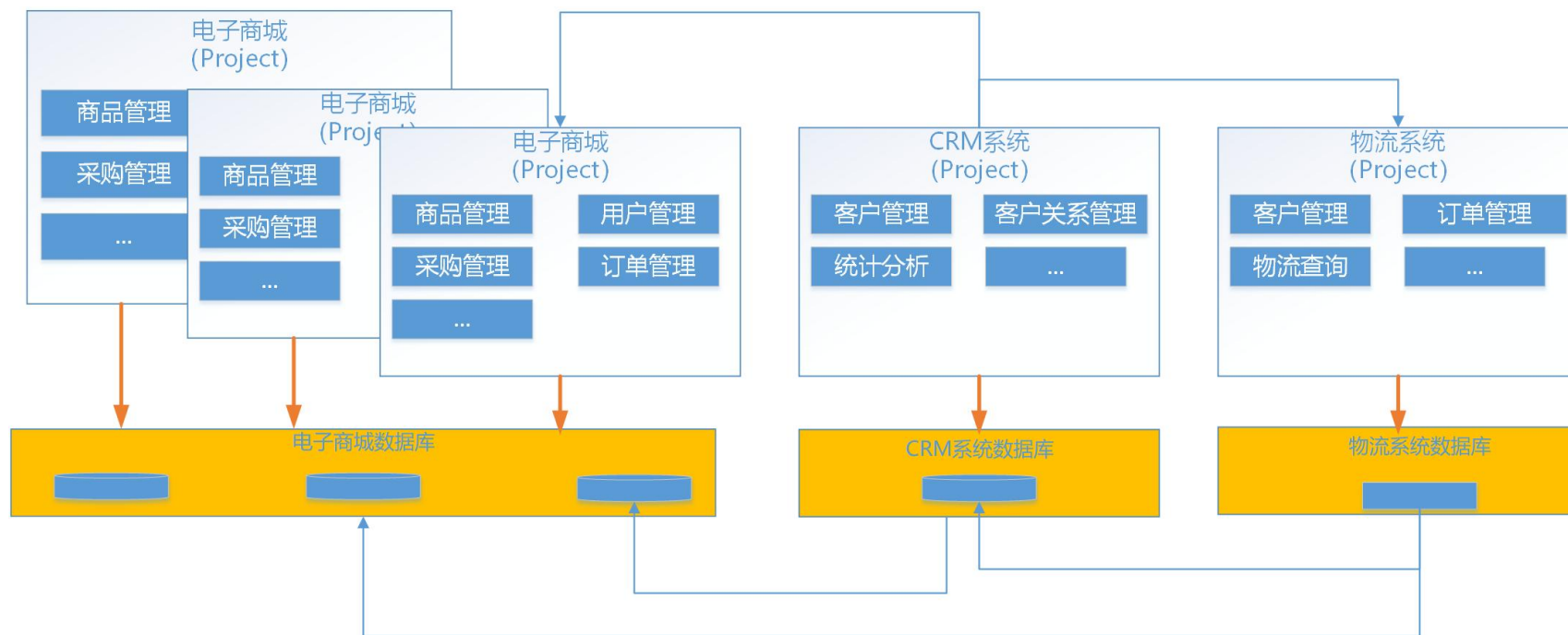
- 1、项目架构简单，前期开发成本低，周期短，小型项目的首选。
- 2、开发效率高，模块之间交互采用本地方法调用。
- 3、容易部署，运维成本小，直接打包为一个完整的包，拷贝到web容器的某个目录下即可运行。
- 4、容易测试：IDE都是为开发单个应用设计的、容易测试——在本地就可以启动完整的系统。

## 缺点：

- 1、全部功能集成在一个工程中，对于大型项目不易开发、扩展及维护。
- 2、版本迭代速度逐渐变慢，修改一个地方就要将整个应用全部编译、部署、启动，开发及测试周期过长。
- 3、无法按需伸缩，通过集群的方式来实现水平扩展，无法针对某业务按需伸缩。

# 分布式架构

针对单体架构的不足，为了适应大型项目的开发需求，许多公司将一个单体系统按业务垂直拆分为若干系统，系统之间通过网络交互来完成用户的业务处理，每个系统可分布式部署，这种架构称为分布式架构。



# 分布式架构

特点：

- 1、按业务垂直拆分成一个一个的单体系统，此架构也称为垂直架构。
- 2、系统与系统之间的存在数据冗余，耦合性较大，如上图中三个项目都存在客户信息。
- 3、系统之间的接口多为实现数据同步，如上图中三个项目要同步客户信息。

优点：

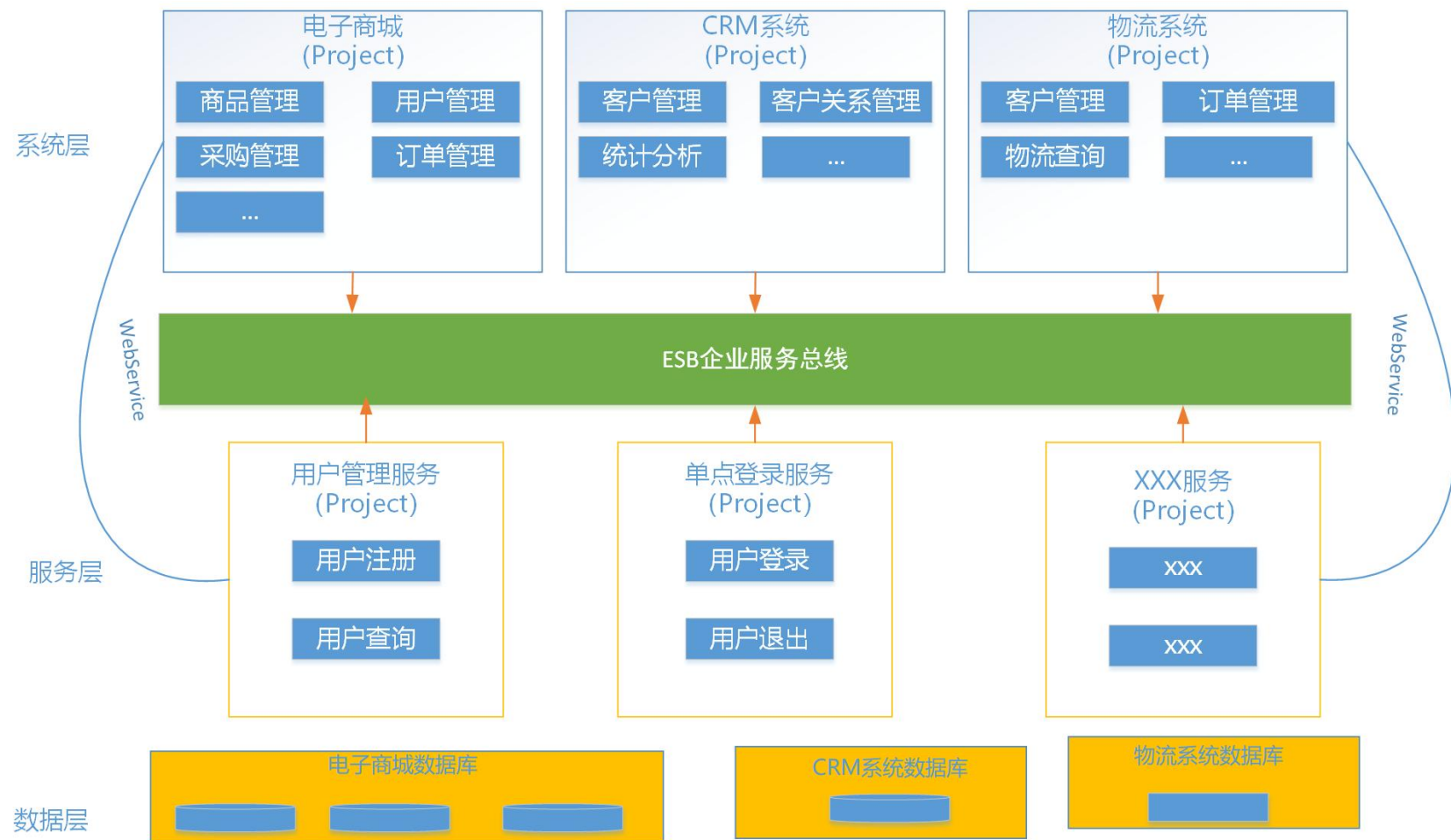
- 1、通过垂直拆分，每个子系统变成小型系统，功能简单，前期开发成本低，周期短。
- 2、每个子系统可按需伸缩。
- 3、每个子系统可采用不同的技术。

缺点：

- 1、子系统之间存在数据冗余、功能冗余，耦合性高。
- 2、按需伸缩粒度不够，对同一个子系统中的不同的业务无法实现，比如订单管理和用户管理。

# SOA架构

SOA是一种面向服务的架构，基于分布式架构，它将不同业务功能按服务进行拆分，并通过这些服务之间定义良好的接口和协议联系起来。



# SOA架构

特点：

- 1、基于SOA的架构思想，将重复公用的功能抽取为组件，以服务的方式向各各系统提供服务。
- 2、各各系统与Service之间采用webservice、rpc等方式进行通信。
- 3、ESB企业服务总线作为系统与Service之间通信的桥梁。

优点：

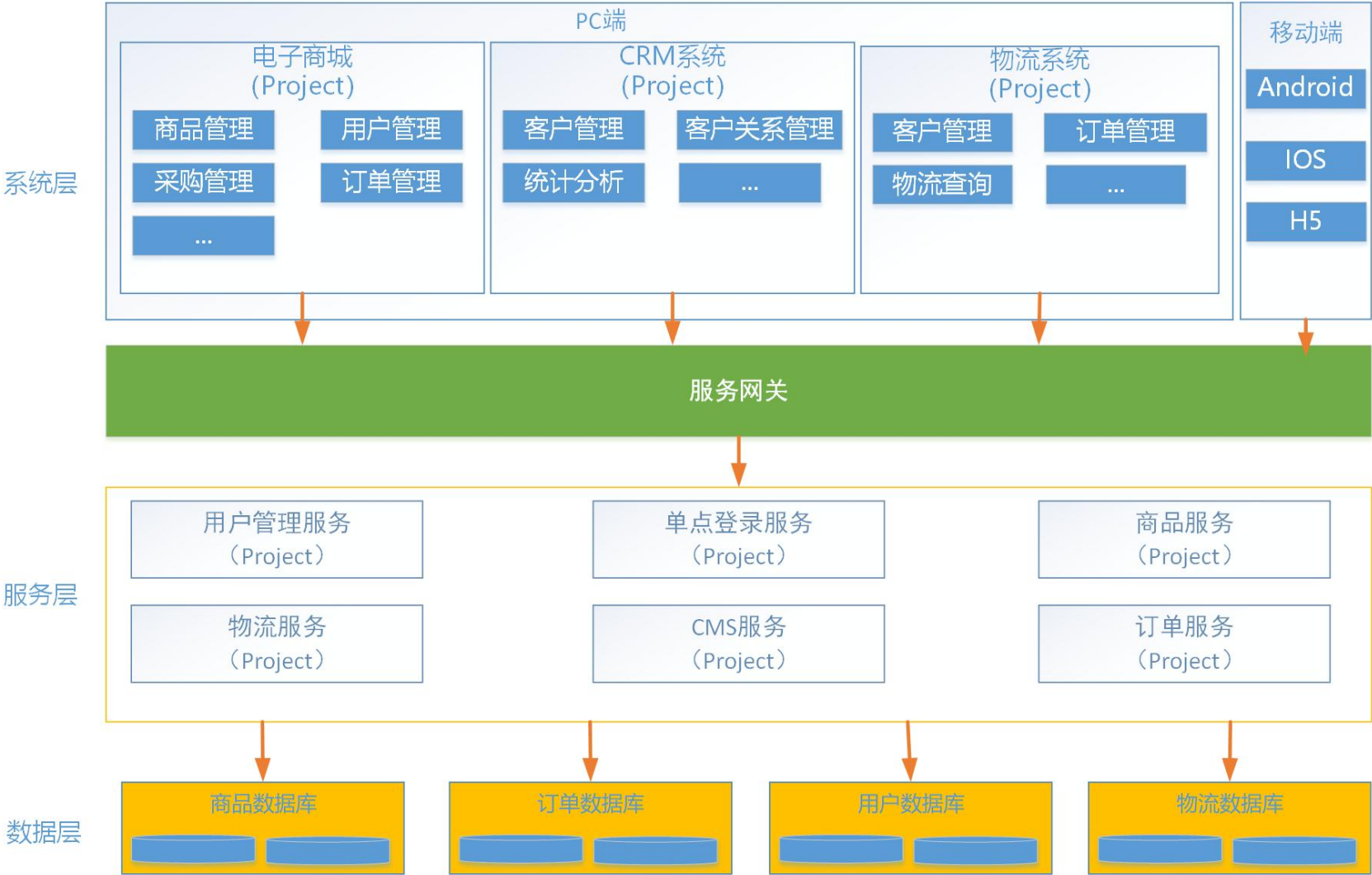
- 1、将重复的功能抽取为Service，提高开发效率，提高系统的可重用性、可维护性。
- 2、可以针对不同Service的特点按需伸缩。
- 3、采用ESB减少系统中的接口耦合。

缺点：

- 1、系统与Service的界限模糊，会导致抽取的Service的粒度过大，系统与Service之间耦合性高。
- 2、虽然使用了ESB，但是Service的接口协议不固定，种类繁多，不利于系统维护。

# 微服务架构

基于SOA架构的思想，为了满足移动互联网对大型项目及多客户端的需求，对服务层进行细粒度的拆分，所拆分的每个服务只完成某个特定的业务功能，比如订单服务只实现订单相关的业务，用户服务实现用户管理相关的业务等等，服务的粒度很小，所以称为微服务架构。



# 微服务架构

## 特点：

- 1、服务层按业务拆分为一个一个的微服务。
- 2、微服务的职责单一。
- 3、微服务之间采用RESTful、RPC等轻量级协议传输。
- 4、有利于采用前后端分离架构。

## 优点：

- 1、服务拆分粒度更细，有利于资源重复利用，提高开发效率。
- 2、可以更加精准的制定每个服务的优化方案，按需伸缩。
- 3、适用于互联网时代，产品迭代周期更短。

## 缺点：

- 1、开发的复杂性增加，因为一个业务流程需要多个微服务通过网络交互来完成。
- 2、微服务过多，服务治理成本高，不利于系统维护。

# 03

---

## 企业级应用架构设计原则

# 设计原则

应用架构设计应遵循五大设计原则：

## 1.业务适配性原则

应用架构要支撑和提升战略能力和业务能力，服务于企业的战略目标。

## 2.企业级原则

应用架构要遵循企业级原则，要打破竖井、整合部门级应用，形成企业级应用，避免功能分散、重叠、界限不清等问题。

## 3.应用组件化原则

要按照业务功能聚合的方式进行应用规划，建设与应用组件对应的应用系统。

## 4.应用松耦合原则

要降低系统间的耦合性，减少相互依赖，降低系统运行的风险。

## 5.资产可复用原则

要沉淀和提炼公共能力，推动架构资产的共享复用，以便于提升开发效率、降低开发维护成本。

# 04

---

## 常见的软件开发模式

# 软件开发模型

## 软件开发模型概述

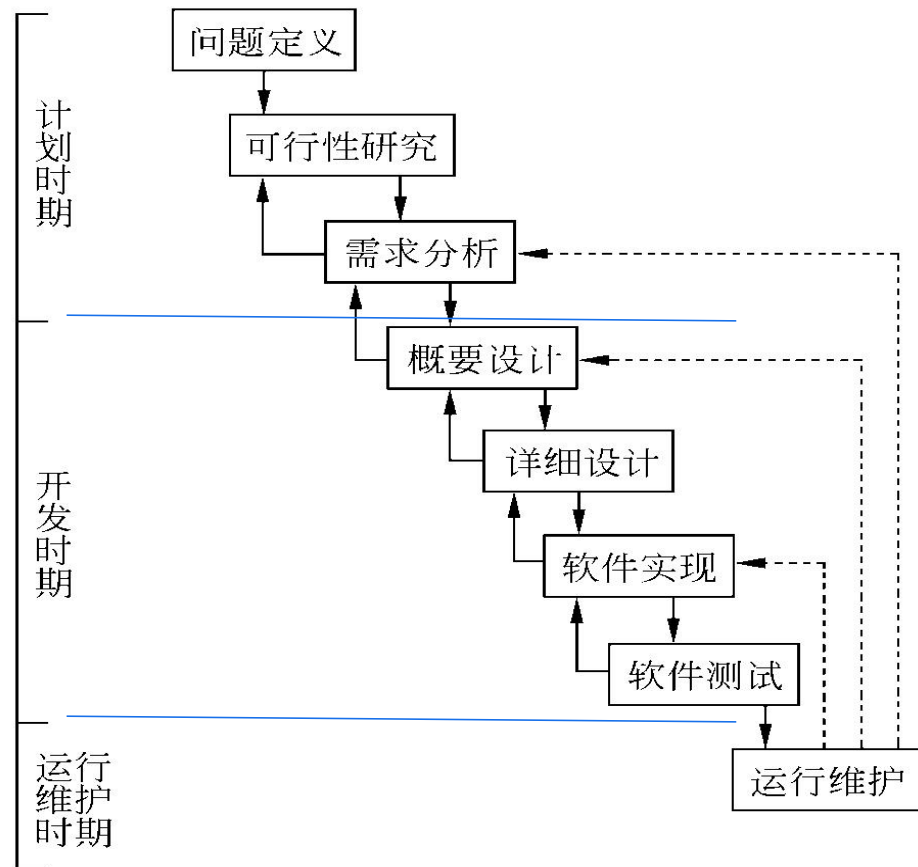
**模型**是对事物本质特征的一种抽象、模拟和描述，用于表示系统的重要特征描述，包括图表模型、数学模型、实物模型和描述模型。根据软件开发工程化及实际需要，软件生命周期的划分不同，形成不同的软件开发模型-软件生命周期模型或软件开发范型（模式）。

## 3-1.瀑布模型

### 瀑布模型概述

**瀑布模型 (Waterfall model)** 将生存期的计划时期、开发时期和运行时期，又细分为若干个阶段：计划时期可分为**问题定义**、**可行性研究**、**需求分析**3个阶段，开发时期分为**概要设计**、**详细设计**、**软件实现**、**软件测试**等阶段，运行时期则需要不断进行运行维护，需要不断修改错误、排除故障，或以用户需求、运行环境改变进行改更调整。右图中的实线箭头表示开发流程，每个阶段顺序进行，有时会返工；虚线箭头表示维护工作的流程，根据不同情况返回到不同的阶段进行维护。

**瀑布模型有很多限制和问题，最主要的是不能拥抱变化**



# 3-1.瀑布模型

## 瀑布模型特点

利用瀑布模型开发软件3个特点:

(1) 开发过程的顺序性。

适用于软件需求明确,开发技术成熟,工程管理较严格情形。

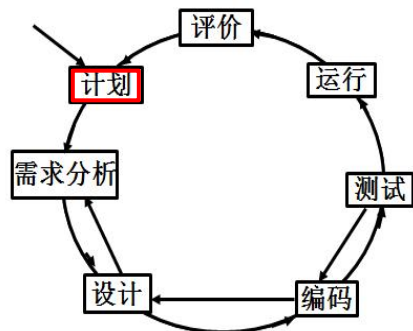
(2) 统筹兼顾不过早编程。

(3) 严格要求保证质量.为确保质量,应坚持做到:

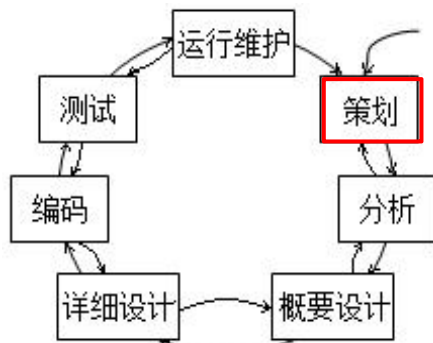
① 必须各阶段都按照要求认真完成规定的文档。

② 各阶段须对完成文档复审,及时发现隐患并排除。

瀑布模型缺陷是将充满回溯且相互重叠的软件开发过程硬性地将分为多个阶段,随着开发软件规模的增加,造成的问题大增。为了描述软件开发过程中可能回溯对瀑布模型进行了改进,开发各阶段可以循环重复。



循环模型

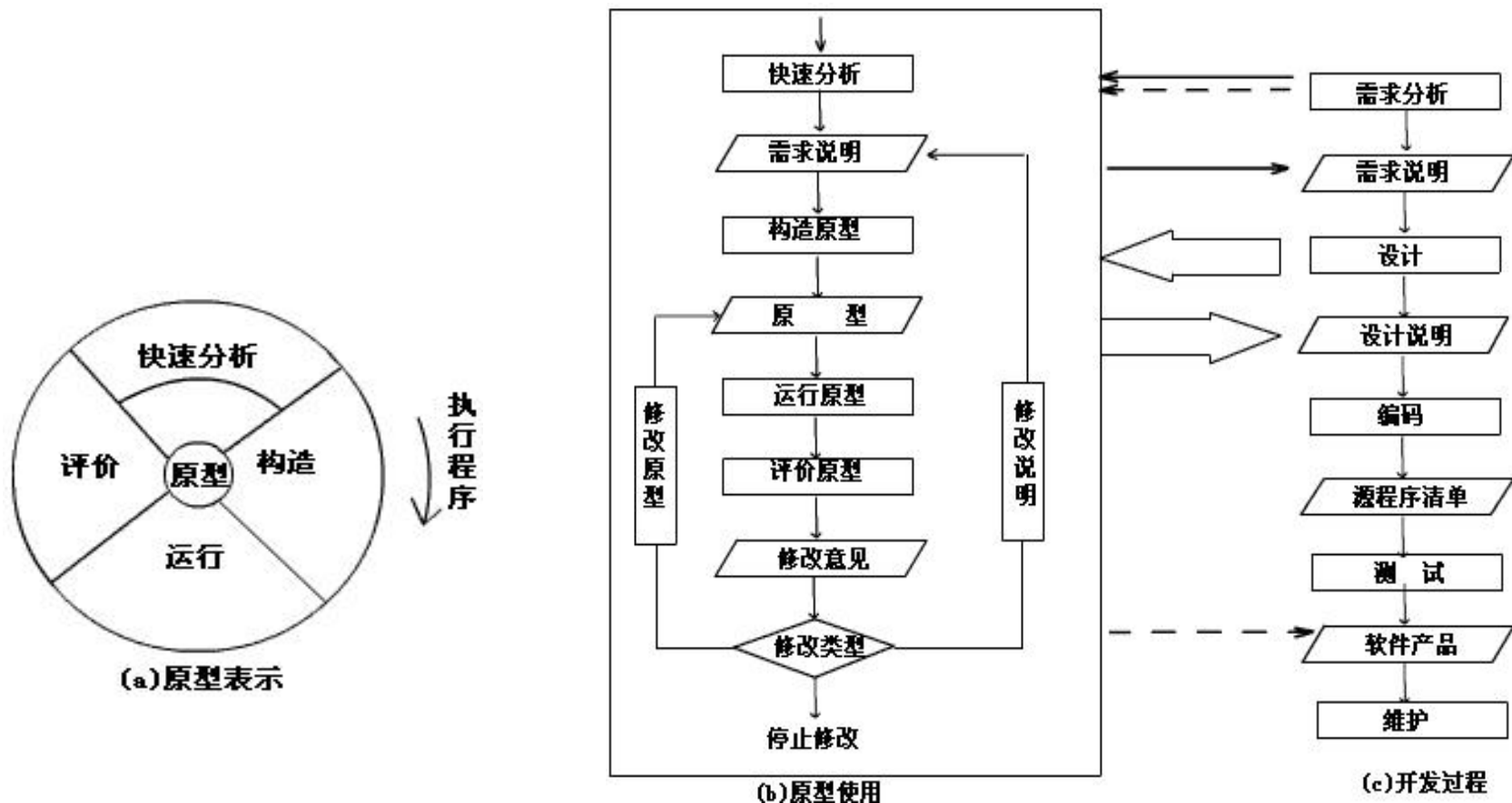


改进模型

## 3-2.快速原型模型

### 快速原型模型

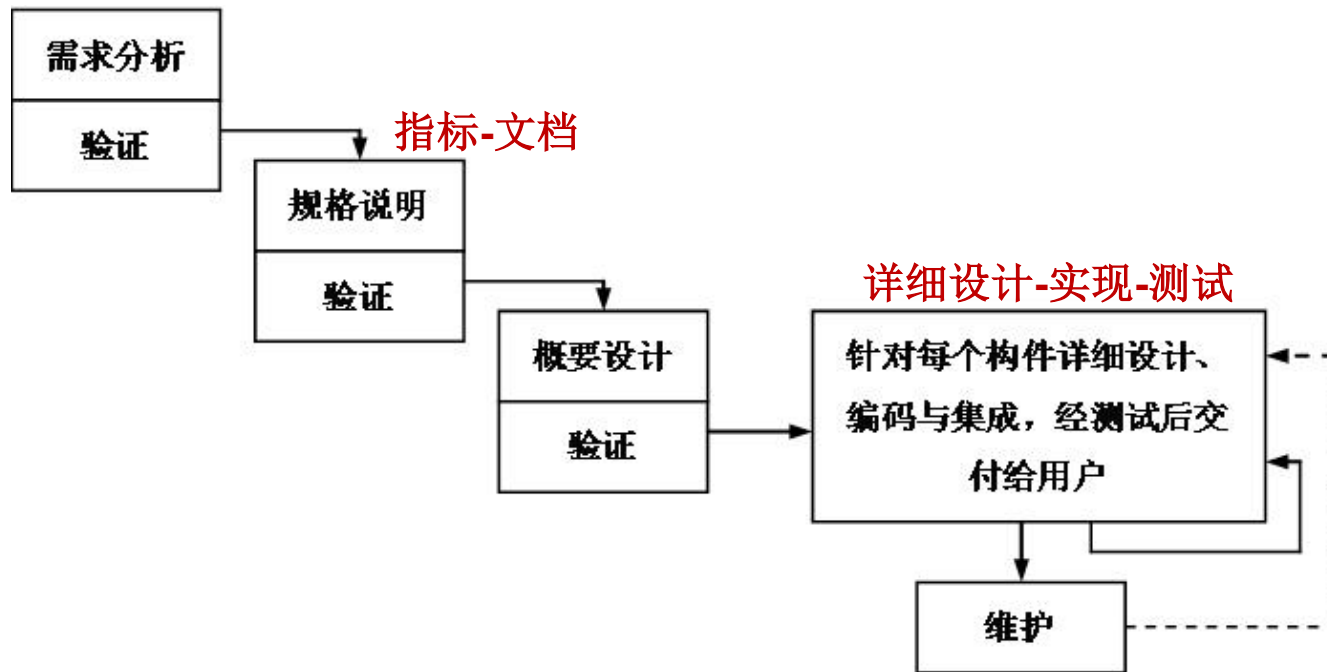
最适合于可以先尽快构建成一个原型的应用系统。需要先建造一个快速原型，如操作窗口及界面等，进行客户或潜在用户与系统间的试用交流，用户/客户可通过对原型的评价及改进意见,进一步细化待开发软件的需求，通过逐步调整原型达到客户要求,从中确定客户的具体需求；然后按照需求开发软件。如图所示。



### 3-3.增量模型

#### 增量模型

增量模型灵活性很强，利用增量模型开发的软件被作为一系列的增量构件进行设计、实现、集成和测试，每个构件具有一定功能，并最终能组合成一个具有完整功能软件的模块。如图所示。适用于软件需求不明确、设计方案有一定风险的软件项目。



## 3-3.增量模型

### 增量模型的缺陷

同瀑布模型的本质区别:瀑布模型属于整体开发模型,规定在开始下一阶段工作之前,必须完成前一阶段的所有细节。而增量模型属于非整体开发模型,可推迟某些阶段或所有阶段中细节,较早地研发出软件。

增量模型的缺陷有两个方面:

- (1) 需要软件具备开放式的体系结构。主要因为各构件要逐渐并入已有的软件体系结构中,所以加入构件不能破坏已构造好的系统部分。
- (2) 软件过程控制易失去整体性.软件在开发中难免需求变化,增量模型的灵活性可使其适应变化的能力优于瀑布模型和快速原型模型,但也容易退化为边做边改模型.

## 3-4.螺旋模型

### 螺旋模型概述

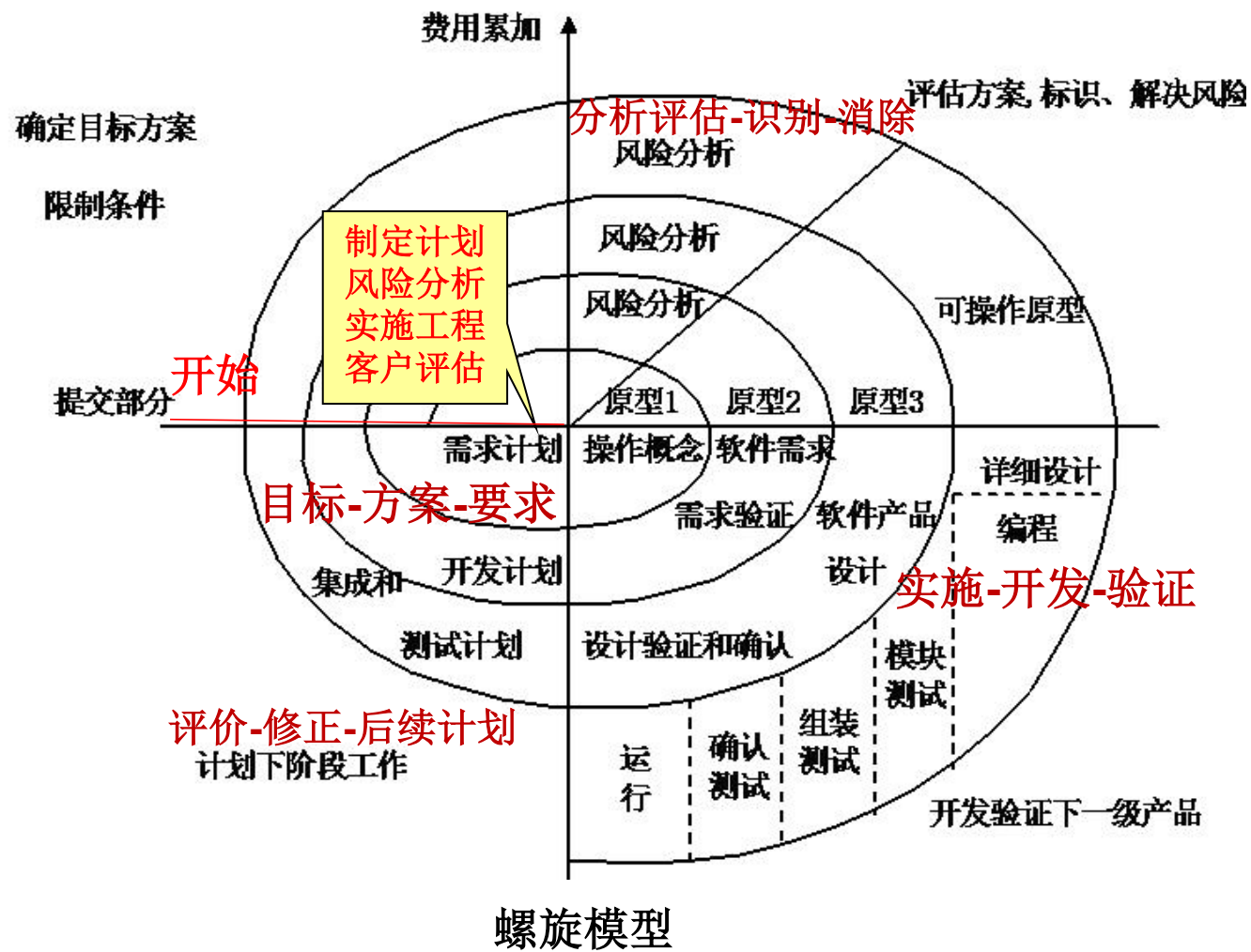
**螺旋模型**将**瀑布模型**和**快速原型模型**结合,强调了其他模型所忽视的风险分析,适合于大型复杂系统,吸收了“演化(推进)”概念,可使开发人员和客户了解对每个演化层的风险,继而做出反应。将开发过程划分为制定计划、风险分析、实施工程和客户评估四类活动,将沿着螺旋线继续进行,自内向外逐步延伸,最终得到满意的软件产品。

螺旋模型沿着螺线进行多次迭代,其迭代过程如下图所示。

- (1) 制定计划：确定软件目标，选定实施方案，弄清项目开发的限制条件；
- (2) 风险分析：分析评估所选方案，考虑如何识别和消除风险；
- (3) 实施工程：实施软件开发和验证；
- (4) 客户评估：评价开发工作，提出修正建议，制定下一步计划。

## 3-4.螺旋模型

### 螺旋模型概述



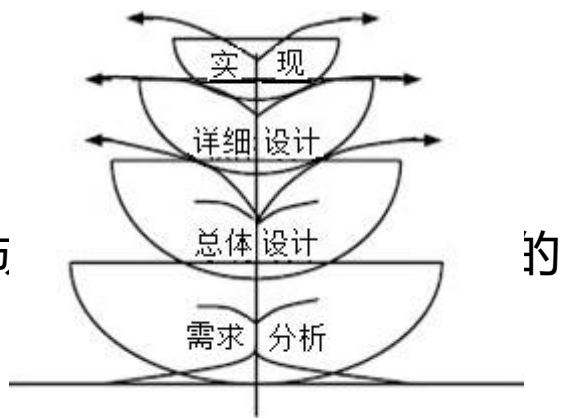
## 3-5.喷泉模型

### 喷泉模型概述及特点

喷泉模型主要适合于利用面向对象技术的软件开发项目。克服了瀑布模型不支持软件重用和多项开发活动集成的局限性。可使开发过程具有迭代性和无间隙性。

喷泉模型是以面向对象的开发方法为基础，以用户需求为源泉。从如图模型中可以看出7个特点：

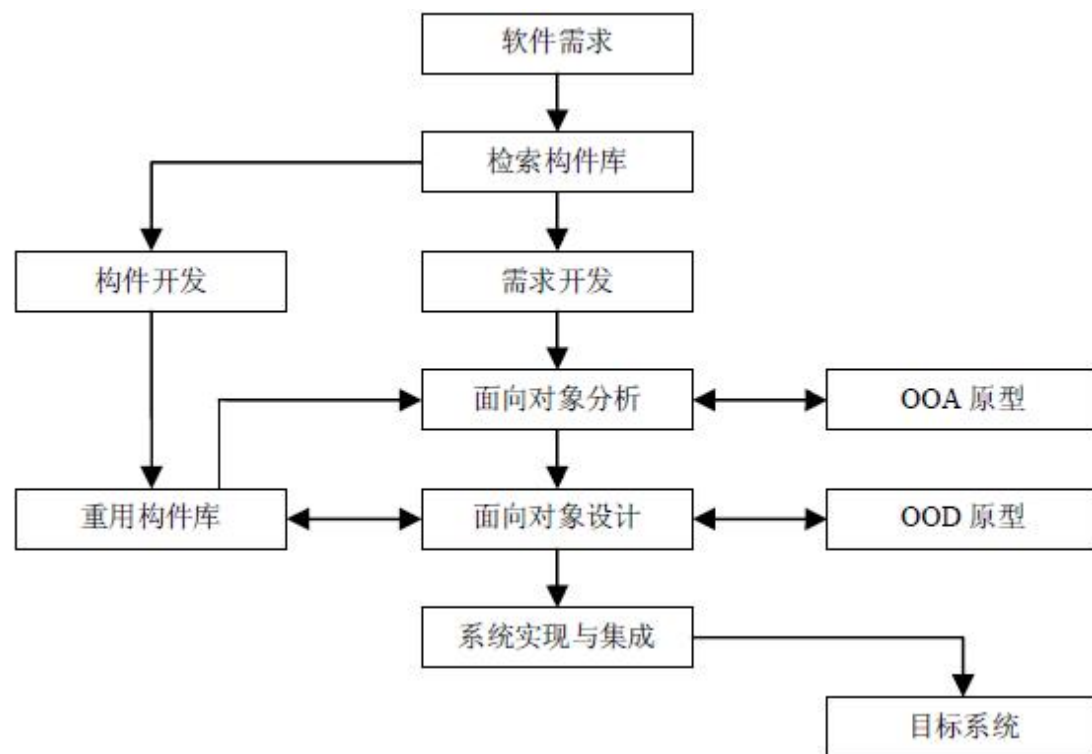
- (1) 规定软件开发过程有4个阶段：需求分析、总体设计、详细设计和实现，还可分成多个开发步骤。
- (2) 各阶段相互重叠，反映了软件过程并行性的特点。
- (3) 以分析为基础，资源消耗成塔形，在分析阶段消耗的资源最多。
- (4) 反映软件过程迭代性的自然特性，从高层返回低层无资源消耗。
- (5) 强调增量开发，依据分析一点设计一点的原则，并不要求一个阶段的彻底完成，逐步提炼的过程。
- (6) 是对象驱动过程，对象是活动作用的实体，也是项目管理的基本内容。
- (7) 实现中由于活动不同，可分为系统实现和对象实现，这既反映了全系统的开发过程，也反映了对象族的开发和重用过程。



## 3-6.基于面向对象的模型

### 基于面向对象的模型

面向对象技术优点很多，应用广泛，构件重用就是其重要技术之一。强调了类的创建与封装，之后便可在不同的应用系统中被重用。面向对象技术为基于构件的软件过程模型提供了强大的技术框架。基于面向对象的模型，综合了面向对象和原型方法及重用技术，模型如下图所示：



基于面向对象的模型

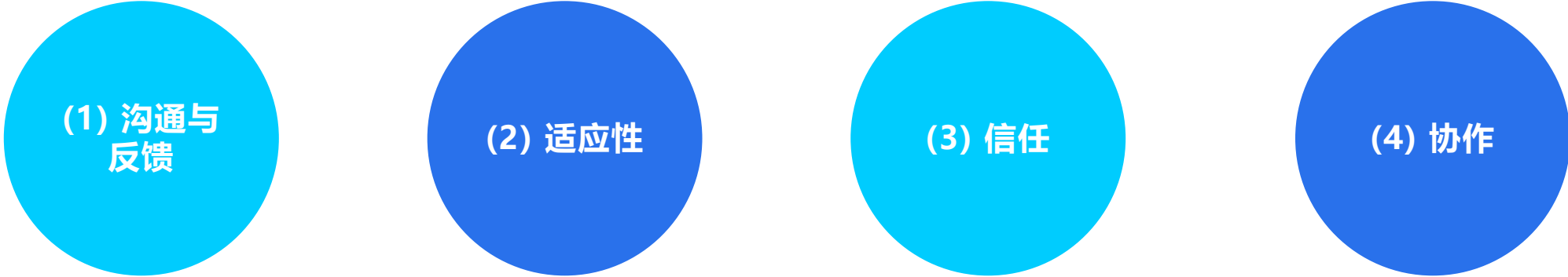
## 3-7.敏捷模型

### 敏捷模型概述及特点

敏捷软件开发又称敏捷开发，是一种从1990年代开始逐渐引起广泛关注的一些新型软件开发方法，是一种**应对快速变化**的需求的一种软件开发能力。它们的具体名称、理念、过程、术语都不尽相同，相对于“非敏捷”，更强调**程序员团队与业务专家**之间的紧密协作、面对面的沟通（认为比书面的文档更有效）、**频繁交付新的软件版本**、紧凑而自我组织型的团队、能够很好地适应**需求变化**的代码编写和团队组织方法，也更注重软件开发中人的作用。

## 3-7.敏捷模型

### 敏捷软件开发的基本原则



#### (1) 沟通与反馈

在使用敏捷方法时，团队成员在同一时间工作，他们可以几乎实时地协作、分享关注点、提供反馈，并高效地协同工作

#### (2) 适应性

创建敏捷软件开发过程时应该考虑的两个基本要求是：一是以恒定的速度推进项目；二是处理压力的能力

#### (3) 信任

选择团队成员使用敏捷方法进行软件开发项目是重要的一个步骤。选择过程的另外重要部分是考虑技能组合和责任，此外还应该有工作环境的个性化匹配。团队成员应该是优秀的专业人士，能够自我管理并且能够相互信任。

#### (4) 协作

在敏捷模型中，客户在开发过程中一直参与以收集需求。因此，客户在维护产品待办事项方面将发挥积极作用。在其他时候，客户可能会在修改需求方面发挥积极作用，例如在提供产品演示、在日常会议期间等等

## 3-8.DevOps

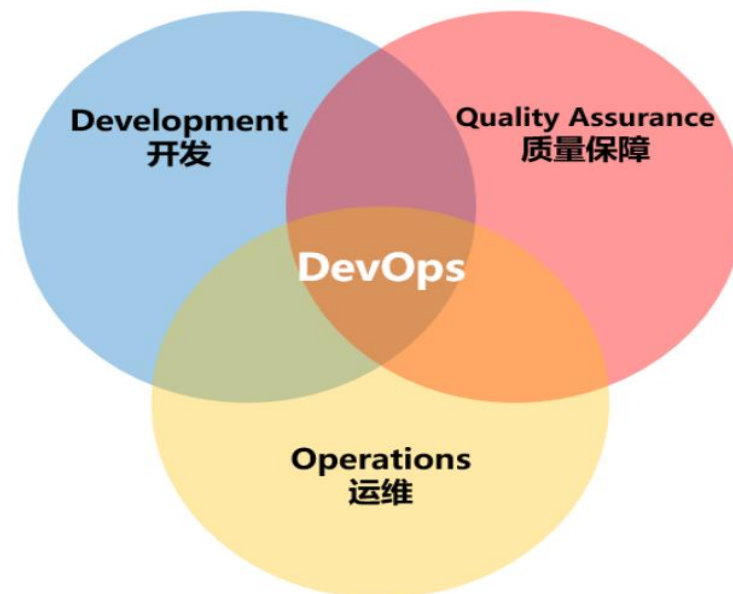
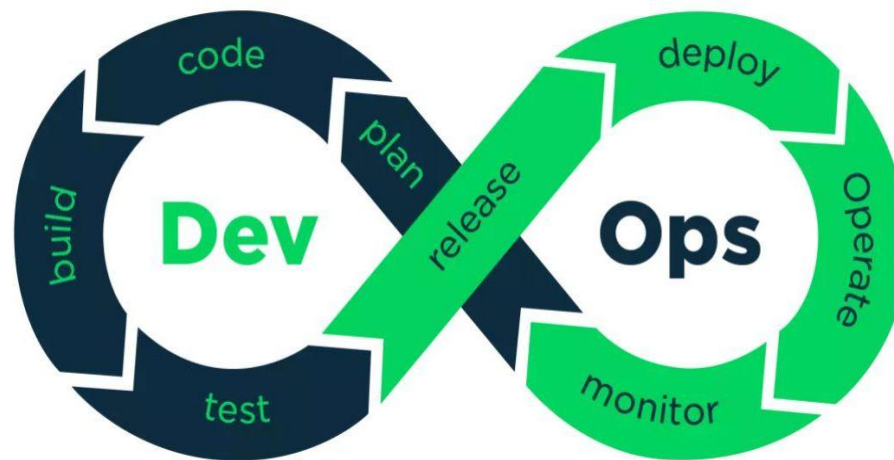
### DevOps 模式概述

DevOps其实就是Development和Operations两个词的组合

DevOps是一组过程、方法与系统的统称，用于促进开发、技术运营和质量保障（QA）部门之间的沟通、协作与整合：

有两个词经常会伴随着DevOps出现，那就是CI和CD。

**CI是Continuous Integration（持续集成）**，而CD对应多个英文，**Continuous Delivery（持续交付）** 或 **Continuous Deployment（持续部署）**

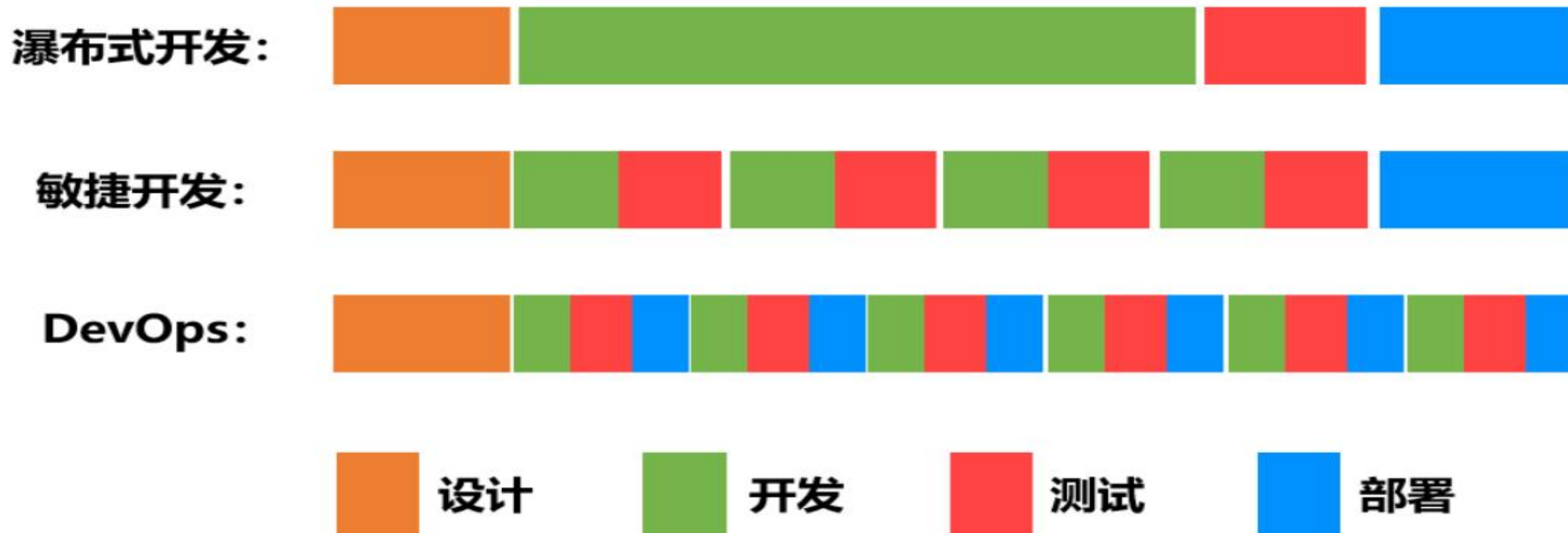


## 3-8.DevOps

### DevOps 模式

DevOps考验的不仅是一家企业的技术，更是管理水平和企业文化。

对比前面所说的瀑布式开发和敏捷开发，我们可以明显看出，DevOps贯穿了软件全生命周期，而不仅限于开发阶段。



## 3-8.DevOps

### DevOps 模式相关概念

**微服务：**不同的工程师可以对各自负责的模块进行处理，例如开发、测试、部署、迭代。

**虚拟化：**其实就是一种敏捷的云计算服务。它从硬件上，将一个系统“划分”为多个系统，系统之间相互隔离，为微服务提供便利。

**容器：**容器就更彻底了，不是划分为不同的操作系统，而是在操作系统上划分为不同的“运行环境”（Container），占用资源更少，部署速度更快。

虚拟化和容器，其实为DevOps提供了很好的前提条件。开发环境和部署环境都可以更好地隔离了，减小了相互之间的影响。



## 3-9. 软件开发模型的选定

### 软件开发模型的选定

最常用的是瀑布模型和原型模型，其次是增量模型，由于迭代模型比较难以掌握使用较少。各种模型各有其特点和优缺点。

在具体选择模型时需要综合考虑6点：

- (1) 符合软件本身的性质，包括规模、复杂性等；
- (2) 满足软件应用系统整体开发进度要求；
- (3) 尽可能控制并消除软件开发风险；
- (4) 具有计算机辅助工具快速的支持，如快速原型工具；
- (5) 同用户和软件开发人员的知识和技能匹配；
- (6) 有利于软件开发的管理与控制。

**注意：**通常情况下，面向过程方法可使用瀑布模型、增量模型和螺旋模型进行开发；面向对象方法可采用快速原型、增量模型、喷泉模型和统一过程进行开发；面向数据方法一般采用瀑布模型和增量模型进行开发。

ありがとう  
感谢 Thanks  
ขอบคุณ  
Terima kasih 谢谢